

PROJECT REPORT

FILE HANDLING APPLICATION

USING PYTHON



A Menu-Driven Console-Based Python Project



PROJECT NAME

File Handling Application



INSTITUTE NAME

Basant Technology



SUBMITTED TO

Ms. Gowthami Shyam



SUBMITTED BY

Priyanka Priyadarshini
Hati



DOS

19th September 2026



GITHUB LINK

<https://github.com/Priyanka-hati/File-Handling-Application>



2. ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Basant Technology for providing me with the opportunity and learning environment required to complete this Python project. This project has helped me understand how the concepts learned during Python training can be combined to create a practical and executable application.

I am thankful to Gowthami Shyam for the guidance, explanations, suggestions, and continuous support provided during the development of the File Handling Application. The trainer's guidance helped me understand important Python topics such as functions, file handling, exception handling, modules, loops, conditional statements, and Object-Oriented Programming (OOP).

Finally, I am grateful to my family and everyone who supported me directly or indirectly while learning Python and completing this project. Their encouragement motivated me to continue learning and complete the project successfully.

This project has been a valuable learning experience because it provided an opportunity to apply Python programming concepts to a real-world type of problem. I hope this report clearly presents the design, implementation, working, and learning outcomes of the project.

3. INDEX

S. No.	Topic	Page No.
1	Cover Page	1
2	Acknowledgement	2
3	Index	3
4	Introduction to Python	4-6
5	Introduction About Project	7-8
6	Concepts Used in Project	9-13
7	Source Code	14-19
8	Working Flow of Code / Explanation	20-21
9	Output	22-28
10	Conclusion	29
11	Bibliography	30

4. INTRODUCTION TO PYTHON

4.1 What is Python?

Python is a high-level, general-purpose programming language known for its simple and readable syntax. It is designed to allow programmers to write programs using statements that are comparatively easy to understand. Python can be used for small scripts as well as large software projects.

Python supports several programming styles, including procedural programming, object-oriented programming, and functional programming. In this project, procedural concepts such as functions and loops are combined with Object-Oriented Programming so that the file operations can be organized inside classes.

One important reason Python is useful for beginners is that the language generally requires less code to perform common tasks. For example, opening a file and reading its content can be done with a few readable statements. Python also provides a large standard library containing modules for operating-system tasks, file manipulation, mathematics, dates, networking, and many other activities.

4.2 History of Python

Python was created by Guido van Rossum and was first released publicly in 1991. The language was designed with an emphasis on readability and developer productivity. Over time, Python gained users in education, software development, automation, scientific computing, data analysis, and many other areas.

Python 2 and Python 3 are major generations of the language. Python 2 was widely used for many years, but it reached its official end of life in 2020. Modern Python development uses Python 3. Python 3 introduced language improvements and became the current standard for new Python applications.

The Python ecosystem has continued to grow through its standard library and third-party packages. This has made Python useful for both beginner projects and professional applications.

4.3 Why Python is Popular

Python is popular because its syntax is comparatively simple, readable, and close to natural programming logic. A beginner can focus on understanding the problem instead of spending excessive time learning complicated syntax.

Python also has a large developer community. There are many learning resources, documentation pages, examples, libraries, and frameworks available. The language is versatile and is used for web development, automation, data science, artificial intelligence, testing, scripting, and many other areas.

For a project such as a File Handling Application, Python is especially convenient because file operations, operating-system operations, strings, exceptions, and user input are supported directly or through the standard library.

4.4 Features of Python

Python has several important features:

- High-level language: Python hides many low-level computer details and allows programmers to work with readable instructions.
- Interpreted language: Python programs are executed by the Python interpreter rather than being handled like a traditional ahead-of-time compiled language.

- Dynamically typed: Variables do not require a fixed type declaration before assignment.
- Object-oriented: Python supports classes, objects, inheritance, encapsulation, and other OOP concepts.
- Open source: Python is available under an open-source license and has a broad community.
- Portable/cross-platform: Python programs can generally run on different operating systems when compatible Python versions and dependencies are available.
- Extensible: Python can work with modules and components written in other languages.
- Large standard library: Python includes many modules for common programming tasks.
- Easy syntax: Python uses a relatively concise and readable syntax.
- Multiple paradigms: Python supports procedural, object-oriented, and functional programming approaches.

4.5 Applications of Python

Python is used in many areas of technology. In web development, frameworks such as Django and Flask are commonly used. In data science, Python is widely used for data cleaning, analysis, visualization, and model development. Python is also widely used in artificial intelligence and machine learning.

In automation, Python can perform repetitive tasks such as file processing, report generation, and system operations. Software testing teams use Python for test automation and API testing. Python is also used for scripting, desktop applications, game development, and API development.

The File Handling Application demonstrates one practical area of Python: automating common file operations through a simple user interface in the console.

4.6 Python Data Types

A data type describes the kind of value stored in a variable. Common Python data types include:

- int – whole numbers such as 10 or -5.
- float – decimal numbers such as 10.5.
- complex – complex numbers such as 2+3j.
- str – text values such as "Hello".
- list – an ordered, mutable collection such as [1, 2, 3].
- tuple – an ordered, immutable collection such as (1, 2, 3).
- set – an unordered collection of unique values such as {1, 2, 3}.
- dictionary – a collection of key-value pairs such as {"name": "Priya"}.
- bool – True or False values.
- NoneType – represented by None and commonly used to indicate the absence of a value.

4.7 Variables and Input

A variable is a name used to store a value. Python creates a variable when a value is assigned to it. For example, `filename = "notes.txt"` stores a file name in the variable `filename`.

The `input()` function allows the program to receive information from the user. The value returned by `input()` is a string. In this project, `input()` is used to receive file names, text content, search words, destination names, choices, and confirmation responses.

Example:

4.8 Conditional Statements

Conditional statements allow a program to make decisions. Python provides if, elif, and else for this purpose.

Example:

4.9 Loops

Loops allow a program to repeat a block of code. A for loop is generally used when iterating through a sequence, while a while loop is useful when a condition should continue to be checked.

The project uses a while loop to keep displaying the menu until the user chooses Exit. A for loop can also be useful for processing lines of a file, although the line-count operation in this project uses readlines() for clarity.

4.10 Functions

A function is a reusable block of code that performs a specific task. Functions make programs easier to understand, test, and maintain.

The application separates file operations into functions such as create_file(), read_file(), append_file(), search_file(), count_lines(), copy_file(), rename_file(), and delete_file(). This modular design prevents all operations from being written inside one large block of code.

4.11 Exception Handling

Programs can encounter errors during execution. For example, a user may try to read a file that does not exist, rename a missing file, or delete a file without permission.

Python provides try and except blocks to handle such situations. Instead of allowing the entire program to stop unexpectedly, the application catches common exceptions and displays a useful message. The project handles FileNotFoundError, FileExistsError, PermissionError, and OSError.

4.12 File Handling in Python

File handling means creating, opening, reading, writing, appending, copying, renaming, and deleting files using a program. Python provides the open() function for working with file objects. The common modes used in this project are r for reading, w for writing or overwriting, and a for appending.

The with open(...) statement is preferred because it automatically manages the file resource and closes the file after the block finishes. The os module is used for operations such as checking whether a file exists, renaming, and deleting. The shutil module is used to copy files.

```
name = input("Enter your name: ")
print("Hello", name)
```

```
age = 18
if age >= 18:
    print("Adult")
elif age > 0:
    print("Minor")
else:
    print("Invalid age")
```

5. INTRODUCTION ABOUT PROJECT

5.1 Project Overview

The File Handling Application is a menu-driven, console-based Python application that allows a user to perform common file operations from one program. The user selects an option from a nine-option menu and enters the required information.

The project is intentionally designed at a beginner-friendly level. It demonstrates how multiple Python concepts can be combined into one practical application. The project uses file handling, functions, classes and objects, inheritance, conditional statements, loops, strings, exception handling, and standard library modules such as `os` and `shutil`.

5.2 Problem Statement

Working with files manually can involve repeated steps such as creating a text file, opening it to read information, adding new content, searching for a word, copying it, changing its name, or removing it. A beginner-friendly program can combine these operations into one menu-driven application.

The problem addressed by this project is therefore the need for a simple console application through which a user can perform common file operations without writing a separate Python program for each operation.

5.3 Project Objectives

The main objectives are to:

- Understand Python file handling.
- Create and write data into files.
- Read existing file content.
- Append new data without replacing existing content.
- Search for a word or phrase.
- Count the number of lines.
- Copy a file to another file name.
- Rename a file.
- Delete a file after confirmation.
- Handle common file-related errors.
- Understand modular programming through functions.
- Understand basic OOP using classes, objects, constructors, encapsulation, and inheritance.

5.4 Scope of Project

The current project focuses on common operations on files, particularly text files, using a console interface. It is intended for learning and demonstration rather than for replacing a full file-management system.

The project can later be expanded with graphical interfaces, folder management, additional file formats, authentication, logging, databases, and cloud storage. These enhancements can make the application more suitable for larger use cases.

5.5 Features of the Application

The application contains exactly nine menu options:

1. Create / Write File – creates a file or overwrites its existing content.

2. Read File – displays the content of an existing file.
3. Append to File – adds new content at the end of a file.
4. Search Word/Phrase in File – checks whether a word or phrase is present.
5. Count Number of Lines – counts the lines in a file.
6. Copy File – creates a copy of the source file.
7. Rename File – changes the file name.
8. Delete File – deletes a file after user confirmation.
9. Exit – closes the application.

5.6 Advantages

The project is simple to operate and easy to understand. It combines multiple basic Python concepts into one practical application. Separate functions make the code modular, while classes organize the file operations. Exception handling provides clearer responses for common errors.

The application also demonstrates how Python's standard library can be used without installing external packages.

5.7 Limitations

The application is console-based and does not provide a graphical user interface. It is mainly intended for text files and does not include a database or user authentication. It also does not provide advanced folder browsing, permissions management, versioning, or cloud synchronization.

5.8 Future Enhancements

Possible future improvements include:

- GUI using Tkinter.
- Folder creation, browsing, and management.
- CSV and JSON support.
- Displaying file size, extension, and modification information.
- Searching across multiple files.
- User authentication.
- Database integration.
- Detailed application logging.
- Cloud storage integration.
- Better validation and a more user-friendly interface.

5.9 Hardware and Software Requirements

Hardware:

- Basic computer or laptop.
- Minimum 4 GB RAM recommended for a comfortable development environment.
- At least 100 MB of free storage for the project and generated sample files.

Software:

- Windows, Linux, or macOS.
- Python 3.x.

- Any Python editor or IDE such as IDLE, VS Code, or PyCharm.
- No external Python package is required for the project.

6. CONCEPTS USED IN PROJECT

6.1 File Handling

File handling is the process of working with files through a program. It includes creating, reading, writing, appending, copying, renaming, and deleting files. In Python, file handling is commonly performed using file objects returned by `open()`.

6.2 `open()` Function

The `open()` function opens a file and returns a file object.

Syntax:

```
file_object = open(file, mode)
```

6.3 File Modes

The mode tells Python what operation is intended for the file.

6.4 Reading a File

The project uses `read()` to retrieve the complete content and `readlines()` to retrieve the file as a list of lines. `readline()` reads one line at a time.

6.5 Writing to a File

`write()` writes a string into a file. `writelines()` can write multiple strings from an iterable. The project uses `write()` because it is easy for a beginner to understand.

6.6 `with open()`

The `with` statement is preferred for file handling because the file is automatically closed when the block finishes, including when an exception occurs during normal context-manager operation.

6.7 Functions

Each file operation is separated into a function. This makes the program modular. A function receives the information it needs, performs its operation, and displays a result.

6.8 Variables

Variables store values such as file names, content, search terms, destination names, user choices, and confirmation responses. Python does not require explicit type declarations for these variables.

6.9 Input and Output

`input()` receives user input from the keyboard. `print()` displays information in the console. Together they provide the user interface for the application.

6.10 Conditional Statements

The main menu uses `if`, `elif`, and `else` to decide which operation should be executed according to the user's selected number.

6.11 While Loop

while True keeps the menu running repeatedly. The application returns to the menu after an operation is completed.

6.12 Break Statement

When the user selects option 9, break is used to leave the while loop. The program then displays the exit message and ends.

6.13 String Operations

The search operation uses lower() so that the comparison is case-insensitive. The in operator checks whether the search term occurs within the file content.

6.14 Exception Handling

The program uses try and except to handle common errors. FileNotFoundError is used when a requested file does not exist. FileExistsError is relevant when a destination already exists. PermissionError handles insufficient permissions. OSError handles other operating-system-related errors.

6.15 os Module

The os module provides operating-system functionality. This project uses os.path.exists() to check whether a file exists, os.rename() to rename a file, and os.remove() to delete a file.

6.16 shutil Module

The shutil module provides high-level file operations. shutil.copy2() is used to copy a file while preserving metadata such as modification time when supported by the operating system.

6.17 Menu-Driven Programming

A menu-driven program presents a list of operations and asks the user to select one. It is useful for beginner applications because the user does not need to remember commands. The project displays the menu repeatedly until Exit is selected.

6.18 Object-Oriented Programming (OOP)

Object-Oriented Programming is a programming approach in which data and related behavior can be organized into classes and objects. Python supports OOP through classes, objects, constructors, instance variables, methods, inheritance, and other features.

In this project, the file operations are grouped into a FileManager class. A second class, FileHandlingApp, inherits from FileManager and manages the menu. This demonstrates how a child class can reuse methods from a parent class.

6.19 Class

A class is a blueprint for creating objects. The FileManager class acts as a blueprint for an object that can perform file operations.

6.20 Object

An object is an instance of a class. In `main()`, the statement `app = FileHandlingApp()` creates an object of the `FileHandlingApp` class.

6.21 Constructor

The `__init__()` method is the constructor used to initialize an object. In the project it stores the application title in an instance variable.

6.22 self

`self` refers to the current object. It is used inside instance methods to access instance variables and other methods.

6.23 Encapsulation

Encapsulation means keeping related data and behavior together. In this project, file-related operations are grouped inside `FileManager` instead of being scattered throughout the application.

6.24 Inheritance

Inheritance allows one class to reuse features of another class. `FileHandlingApp(FileManager)` means `FileHandlingApp` is the child class and `FileManager` is the parent class. The child class can use the file-operation methods defined in the parent class.

6.25 Polymorphism

Polymorphism means that a common interface can represent different implementations. Python supports polymorphism in several ways. The present project does not require a complex polymorphism example; the focus is on practical class use and inheritance. This keeps the project beginner-friendly rather than adding unnecessary complexity.

6.26 OOP Implementation in This Project

The OOP structure is:

- `FileManager` – parent class containing `create`, `read`, `append`, `search`, `count`, `copy`, `rename`, and `delete` methods.
- `FileHandlingApp` – child class that inherits `FileManager` and provides the menu and main application flow.
- `__init__()` – initializes the application title.
- Object – `app = FileHandlingApp()` creates the application object.
- Encapsulation – file operations remain grouped inside the class.
- Inheritance – the child class reuses all file-operation methods from the parent class.

```
open(filename, mode)
```

Mode	Meaning	Usage
r	Read	Read an existing file
w	Write	Create a file or overwrite existing content

a	Append	Add content at the end of a file
x	Create	Create a new file; raises an error if it already exists

Additional modes: r+ opens for reading and writing, w+ opens for writing and reading (and can overwrite existing content), and a+ opens for appending and reading.

```
with open(filename, "r") as file:
    content = file.read()
```

```
with open(filename, "w") as file:
    file.write(content)
```

```
if word.lower() in content.lower():
```

```
os.path.exists(filename)
os.rename(old_name, new_name)
os.remove(filename)
```

```
shutil.copy2(source, destination)
```

```
while True:
    # display menu
    ...
    if choice == "9":
        break
```

7. SOURCE CODE

The following is the complete executable source code. It keeps the requested nine menu options, uses the requested operation names, uses os and shutil, includes exception handling, and adds basic OOP through a parent FileManager class and a child FileHandlingApp class.

```
[3]: import os
import shutil

print("Modules imported successfully.")

Modules imported successfully.

[9]: # =====
# PARENT CLASS: FileManager
# =====

class FileManager:

    # =====
    # CONSTRUCTOR
    # =====
    def __init__(self):
        self.app_name = "FILE HANDLING APPLICATION"

    # =====
    # OPTION 1: CREATE / WRITE FILE
    # =====
    def create_file(self):

        try:
            filename = input("Enter file name: ").strip()
            content = input("Enter content: ")

            with open(filename, "w") as file:
                file.write(content + "\n")

            # Get the complete location of the file
            file_path = os.path.abspath(filename)

            print("File created/written successfully.")
            print("File stored at:", file_path)

        except PermissionError:
            print("Error: Permission denied.")

        except OSError as e:
            print("Error:", e)

    # =====
    # OPTION 2: READ FILE
    # =====
    def read_file(self):

        try:
            filename = input("Enter file name: ").strip()

            with open(filename, "r") as file:
                content = file.read()

            print("\n---- File Content ----")
            print(content, end="")
            print("\n-----")

            file_path = os.path.abspath(filename)
            print("File location:", file_path)

        except FileNotFoundError:
            print("Error: File not found.")

        except PermissionError:
            print("Error: Permission denied.")

        except OSError as e:
            print("Error:", e)
```

```

# -----
# OPTION 3: APPEND TO FILE
# -----
def append_file(self):

    try:
        filename = input("Enter file name: ").strip()
        content = input("Enter content to append: ")

        with open(filename, "a") as file:
            file.write(content + "\n")

        file_path = os.path.abspath(filename)

        print("Content appended successfully.")
        print("File location:", file_path)

    except FileNotFoundError:
        print("Error: File not found.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

# -----
# OPTION 4: SEARCH IN FILE
# -----
def search_file(self):

    try:
        filename = input("Enter file name: ").strip()
        word = input("Enter word/phrase to search: ").strip()

        with open(filename, "r") as file:
            content = file.read()

        if word.lower() in content.lower():
            print("Word/phrase found in the file.")
        else:
            print("Word/phrase not found in the file.")

        file_path = os.path.abspath(filename)
        print("File location:", file_path)

    except FileNotFoundError:
        print("Error: File not found.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

```

```

# =====
# OPTION 5: COUNT LINES
# =====
def count_lines(self):

    try:
        filename = input("Enter file name: ").strip()

        with open(filename, "r") as file:
            lines = file.readlines()

        print("Number of lines:", len(lines))

        file_path = os.path.abspath(filename)
        print("File location:", file_path)

    except FileNotFoundError:
        print("Error: File not found.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

# =====
# OPTION 6: COPY FILE
# =====
def copy_file(self):

    try:
        source = input("Enter source file name: ").strip()
        destination = input("Enter destination file name: ").strip()

        if os.path.exists(destination):
            raise FileExistsError

        shutil.copy2(source, destination)

        destination_path = os.path.abspath(destination)

        print("File copied successfully.")
        print("Copied file stored at:", destination_path)

    except FileNotFoundError:
        print("Error: Source file not found.")

    except FileExistsError:
        print("Error: Destination file already exists.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

```

```

# -----
# OPTION 7: RENAME FILE
# -----
def rename_file(self):

    try:
        old_name = input("Enter current file name: ").strip()
        new_name = input("Enter new file name: ").strip()

        if not os.path.exists(old_name):
            raise FileNotFoundError

        if os.path.exists(new_name):
            raise FileExistsError

        os.rename(old_name, new_name)

        new_path = os.path.abspath(new_name)

        print("File renamed successfully.")
        print("File stored at:", new_path)

    except FileNotFoundError:
        print("Error: Current file not found.")

    except FileExistsError:
        print("Error: New file name already exists.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

# -----
# OPTION 8: DELETE FILE
# -----
def delete_file(self):

    try:
        filename = input("Enter file name: ").strip()

        if not os.path.exists(filename):
            raise FileNotFoundError

        file_path = os.path.abspath(filename)

        confirmation = input(
            "Are you sure you want to delete it? (yes/no): "
        ).strip().lower()

        if confirmation == "yes":
            os.remove(filename)

            print("File deleted successfully.")
            print("Deleted file was located at:", file_path)

        else:
            print("Delete operation cancelled.")

    except FileNotFoundError:
        print("Error: File not found.")

    except PermissionError:
        print("Error: Permission denied.")

    except OSError as e:
        print("Error:", e)

```

```
[10]: # =====
# CHILD CLASS: FileHandlingApp
# =====

class FileHandlingApp(FileManager):

    # =====
    # CONSTRUCTOR
    # =====
    def __init__(self):
        super().__init__()

    # =====
    # MAIN MENU
    # =====
    def main(self):

        while True:

            print("\n-----")
            print("      FILE HANDLING APPLICATION")
            print("-----")
            print("1. Create / Write File")
            print("2. Read File")
            print("3. Append to File")
            print("4. Search in File")
            print("5. Count Lines")
            print("6. Copy File")
            print("7. Rename File")
            print("8. Delete File")
            print("9. Exit")
            print("-----")

            choice = input("Enter your choice: ").strip()

            # =====
            # CHOICE 1: CREATE / WRITE
            # =====
            if choice == "1":

                print("\n--- CREATE / WRITE FILE ---")
                self.create_file()

            # =====
            # CHOICE 2: READ
            # =====
            elif choice == "2":

                print("\n--- READ FILE ---")
                self.read_file()

            # =====
            # CHOICE 3: APPEND
            # =====
            elif choice == "3":

                print("\n--- APPEND TO FILE ---")
                self.append_file()

            # =====
            # CHOICE 4: SEARCH
            # =====
            elif choice == "4":

                print("\n--- SEARCH IN FILE ---")
                self.search_file()
```

```

# -----
# CHOICE 4: SEARCH
# -----
elif choice == "4":

    print("\n--- SEARCH IN FILE ---")
    self.search_file()

# -----
# CHOICE 5: COUNT LINES
# -----
elif choice == "5":

    print("\n--- COUNT LINES ---")
    self.count_lines()

# -----
# CHOICE 6: COPY
# -----
elif choice == "6":

    print("\n--- COPY FILE ---")
    self.copy_file()

# -----
# CHOICE 7: RENAME
# -----
elif choice == "7":

    print("\n--- RENAME FILE ---")
    self.rename_file()

# -----
# CHOICE 8: DELETE
# -----
elif choice == "8":

    print("\n--- DELETE FILE ---")
    self.delete_file()

# -----
# CHOICE 9: EXIT
# -----
elif choice == "9":

    print("\nThank you for using the application.")
    break

# -----
# INVALID CHOICE
# -----
else:

    print("\nInvalid choice.")
    print("Please enter a number from 1 to 9.")

```

```

[*]: # -----
# START FILE HANDLING APPLICATION
# -----

app = FileHandlingApp()
app.main()

```



7.1 Source Code Structure

The FileManager class contains the actual file-operation methods. FileHandlingApp inherits from FileManager and controls the menu. The small standalone functions with the requested names are included as simple wrappers so that the report also demonstrates function-based modular programming. The application's normal execution starts through main(), which creates a FileHandlingApp object.

8. WORKING FLOW OF CODE

8.1 Program Execution Flow

```
Start
↓
Import Modules
↓
Create FileHandlingApp Object
↓
Display Menu
↓
Take User Choice
↓
Execute Selected Method
↓
Display Result
↓
Return to Menu
↓
Choice = 9?
↓
Yes → Exit
No → Display Menu Again
```

8.2 Explanation of Each Function / Method

Operation	Purpose	Input	Processing	Output	Concepts
1. create_file()	Creates a new file or overwrites an existing file using w mode. It asks for the file name and content, writes the content, and displays a success message.	File name and content.	Opens the file with with open(..., 'w'), writes the content, and closes the file automatically.	Success message or an error message.	Functions, strings, input(), with open(), write(), exceptions, OOP method.
2. read_file()	Reads and displays an existing file.	File name.	Opens the file using r mode and reads its complete content with read().	The file content.	File handling, read(), input(), exceptions.
3. append_file()	Adds new content to the end of an existing file without replacing existing content.	File name and content to append.	Opens the file in a mode and writes the new line at the end.	Success message.	a mode, write(), input(), exceptions.
4. search_file()	Searches for a word or phrase	File name and search	Reads the file, converts both	Found/not found message.	Strings, lower(), in, read(),

	without treating uppercase and lowercase as different.	word/phrase.	strings to lowercase, and uses the in operator.		conditionals.
5. count_lines()	Counts the number of lines in a file.	File name.	Reads all lines using readlines() and calculates len(lines).	Number of lines.	readlines(), list, len(), exceptions.
6. copy_file()	Copies a source file to a new destination file name.	Source and destination names.	Checks whether the destination exists and then uses shutil.copy2().	Success or error message.	os.path.exists(), shutil.copy2(), exceptions.
7. rename_file()	Changes the name of an existing file.	Old and new file names.	Checks both names and uses os.rename().	Success or error message.	os.path.exists(), os.rename(), exceptions.
8. delete_file()	Deletes a file after asking the user for confirmation.	File name and yes/no confirmation.	Checks existence, asks for confirmation, and uses os.remove() only when the answer is yes.	Success or cancellation/error message.	input(), conditionals, os.remove(), exceptions.
9. main()	Starts the application by creating the child-class object and calling its menu method.	No direct file input; menu input is handled inside the application.	Creates FileHandlingApp() and calls main().	Runs the menu until Exit.	Objects, inheritance, while loop, if/elif/else, break.

8.3 Explain Important Code Statements

with open(filename, "r") as file:

Opens the file in read mode and stores the file object in the variable file. The with statement manages the file resource automatically.

content = file.read()

Reads the complete content of the opened file and stores it in content.

file.write(content)

Writes the string stored in content into the file. In the actual project, the entered content is written with a newline.

if word.lower() in content.lower():

Converts both the search term and file content to lowercase and checks whether the search term occurs in the content.

os.path.exists(filename)

Checks whether the specified path exists.

os.rename(old_name, new_name)

Changes the file name from old_name to new_name.

os.remove(filename)

Removes the specified file from the file system.

shutil.copy2(source, destination)

Copies the source file to the destination while attempting to preserve file metadata.

while True:

Creates a loop that keeps the menu active until break is executed.

break

Immediately exits the menu loop when the user selects option 9.

class FileHandlingApp(FileManager):

Defines a child class that inherits methods from FileManager.

app = FileHandlingApp()

Creates an object of the child class.

super().__init__()

Calls the parent class constructor so that parent initialization is performed.

9. OUTPUT

The following sample outputs use one logical sequence. Option 1 creates notes.txt with one line. Option 3 appends a second line. Therefore, when Option 2 reads the file and Option 5 counts its lines, the output reflects the two lines. Option 6 copies notes.txt to backup.txt, Option 7 renames backup.txt to notes_backup.txt, and Option 8 deletes notes_backup.txt after confirmation.

9.1 Option 1 – Create / Write File

Purpose: Create File Handling Project.txt and write the first line.

Input: File name = File Handling Project.txt; content = Name of the Project : File Handling Application using basic Python.

Expected Console Output

```
=====
      FILE HANDLING APPLICATION
=====
1. Create / Write File
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 1

--- CREATE / WRITE FILE ---
Enter file name: File Handling Project.txt
Enter content: File Handling Application using basic Python.
File created/written successfully.
File stored at: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project.txt
```

9.2 Option 2 – Read File

Purpose: Display the current content of File Handling Project.txt after it has been created and before the append operation.

Expected Console Output

```
=====
FILE HANDLING APPLICATION
=====
1. Create / Write File
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 2

--- READ FILE ---
Enter file name: File Handling Project.txt

----- File Content -----
File Handling Application using basic Python.

-----
File location: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project.txt
```

9.3 Option 3 – Append to File

Purpose: Add a second line without replacing the first line.

Input: File name = File Handling Project.txt; content = Project Created By: Priyanka Priyadarshini Hati

Expected Console Output

```
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 3

--- APPEND TO FILE ---
Enter file name: File Handling Project.txt
Enter content to append: Project Created By: Priyanka Priyadarshini Hati
Content appended successfully.
File location: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project.txt
```

9.4 Option 4 – Search in File

Purpose: Search for the phrase Created by in the File Handling Project.txt file.

Expected Console Output

```
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 4

--- SEARCH IN FILE ---

Enter file name: File Handling Project.txt
Enter word/phrase to search: Created by
Word/phrase found in the file.
File location: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project.txt
```

9.5 Option 5 – Count Lines

Purpose: Count the two lines currently stored in File Handling Project.txt.

Expected Console Output

```
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 5

--- COUNT LINES ---

Enter file name: File Handling Project.txt
Number of lines: 2
File location: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project.txt
```

9.6 Option 6 – Copy File

Purpose: Copy File Handling Project.txt to File Handling Project backup.txt.

Expected Console Output

```
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 6

--- COPY FILE ---

Enter source file name: File Handling Project.txt
Enter destination file name: File Handling Project Backup.txt
File copied successfully.
Copied file stored at: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File Handling Project Backup.txt
```

9.7 Option 7 – Rename File_

Purpose: Rename File Handling Project backup.txt to File_Handling_Project_backup.txt.

Expected Console Output

```
=====
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 7

--- RENAME FILE ---

Enter current file name: File Handling Project backup.txt
Enter new file name: File_Handling_Project_backup.txt
File renamed successfully.
File stored at: C:\Users\priya\Documents\Python\PYTHON PROJECT\File Handling Project backup.txt
```

9.8 Option 8 – Delete File

Purpose: File_Handling_Project_backup.txt after confirmation.

Expected Console Output

```
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 8

--- DELETE FILE ---
Enter file name: File_Handling_Project_backup.txt
Are you sure you want to delete it? (yes/no): yes
File deleted successfully.
Deleted file was located at: C:\Users\priya\Documents\Python\PYTHON_PROJECT\File_Handling_Project_backup.txt
```

9.9 Option 9 – Invalid File

Purpose: If entered file is not available.

Expected Console Output

```
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice: 7

--- RENAME FILE ---
Enter current file name: File Handling Project backup
Enter new file name: File_Handling_Project_backup.txt
Error: Current file not found.
```

9.10 Option 10 – Exit

Purpose: End the application.

Expected Console Output

```
=====
      FILE HANDLING APPLICATION
=====
1. Create / Write File
2. Read File
3. Append to File
4. Search in File
5. Count Lines
6. Copy File
7. Rename File
8. Delete File
9. Exit
=====
Enter your choice:  9

Thank you for using the application.
```

10. CONCLUSION

The File Handling Application using Python was developed as a practical project to demonstrate how basic Python concepts can be combined to solve a real-world type of problem. The application provides nine menu options for creating, reading, appending, searching, counting lines, copying, renaming, deleting, and exiting.

During the development of the project, important concepts such as variables, input and output, strings, conditional statements, loops, functions, file modes, `read()`, `readlines()`, `write()`, exception handling, and standard library modules were applied. The project also introduces Object-Oriented Programming. The `FileManager` class groups the file-related behavior, while `FileHandlingApp` inherits from it and manages the application menu. The constructor, `self`, object creation, encapsulation, and inheritance provide a practical introduction to OOP.

File handling is important because many applications need to store and retrieve information from files. Although this project is intentionally simple and console-based, it demonstrates the basic operations required to work with text files safely and systematically. Exception handling also improves the user experience by providing understandable responses when a file is missing, already exists, cannot be accessed, or produces another operating-system-related error.

The project can be enhanced in the future by adding a graphical interface, folder management, support for CSV and JSON files, authentication, logging, database integration, and cloud storage. These improvements would extend the application beyond a learning project and could make it more useful for larger workflows.

Overall, the project provides practical experience with Python programming, file management, modular design, error handling, and basic OOP. It also provides a strong foundation for understanding how individual Python concepts work together inside an executable application.

11. BIBLIOGRAPHY

The following sources are recommended for further reading and verification of Python language and standard-library concepts:

- Gowthami Shyam Madam's explanations and practical exercises.
- Course notes and classroom materials provided by Basant Technology.
- Python Software Foundation – Python Documentation: <https://docs.python.org/3/>
- Python Documentation – Built-in Functions / open():
<https://docs.python.org/3/library/functions.html#open>
- Python Documentation – Input and Output / File Objects:
<https://docs.python.org/3/tutorial/inputoutput.html>
- Python Documentation – os module: <https://docs.python.org/3/library/os.html>
- Python Documentation – shutil module: <https://docs.python.org/3/library/shutil.html>
- Python Documentation – Classes and Objects: <https://docs.python.org/3/tutorial/classes.html>
- Python Documentation – Errors and Exceptions: <https://docs.python.org/3/tutorial/errors.html>

